C03 | 怎样让坏数据进不来

C01、C02 判断答案；C03 反过来设计**事实放哪里、怎样对应、哪些坏状态不能保存**。约束能保护写入入口，但不会替应用补齐所有业务判断。

哪些事实需要独立身份与保存位置？

当前问题	关系或短用法	记到什么程度
本章预约怎样找到客户和技术员？	客户经工单找到，技术员经时段找到：customers ← tickets ← appointments → slots → technicians。箭头从引用行指向被引用行。	**能沿真实外键追事实。** 工单未预约、时段未被选择时也能独立存在，不照抄前章的小表列。
主键和外键各保护什么？	主键让 ID 唯一确定本表行；`customer_id REFERENCES customers(id)` 要求非空引用找到目标客户。	姓名可重名和改变，不能替代这里的稳定身份；C999 失败是目标不存在，不是编号格式不对。
多对多为什么用组合主键？	`PRIMARY KEY (technician_id, skill_id)` 拒绝重复 `(T1,S1)`，仍允许 `(T1,S2)` 与 `(T2,S1)`。	认熟“唯一的是一对关系”，不误读成每个技术员只能出现一次。
规范化在这里实际解决什么？	客户当前邮箱在客户表，技能名称在技能表；其他行通过身份找它，减少同一事实多处更新的矛盾。	能解释拆表理由；历史发信所用地址是另一种事实，不该冒充当前邮箱，也不必把每个文字都拆表。

完整七表 `c_schema.sql` 的价值在共同表达这些关系。表名与全部列无需背诵；需要改结构时，先找独立对象和引用方向，再查看整份 schema。

回看：「一张预约究竟连接了哪些事实？」「数据库怎样发现 C999 根本不存在？」「技能为什么还需要一张关系表？」「把这些决定放在一起，表应该怎样建？」。

哪一种坏值，应由哪条规则拒绝？

当前问题	关系或短用法	记到什么程度
写了 INTEGER，为何仍收到文字或负数？	普通 SQLite 表有类型亲和性；`'60'` 可转成整数，`'thirty'` 可留下 text，负数也是整数。	**类型声明与业务规则分开。** typeof(列) 可看存储类别；不由列名或声明推断原始输入类型。
时长必须是正整数，怎样表达？	`INTEGER NOT NULL CHECK (typeof( duration_minutes) = 'integer' AND duration_minutes > 0)`。	能指出非空、存储类别、正数各守哪一处；完整建列写法查用。转换后的 `'60'` 仍能通过。
必填、唯一、范围与真实引用如何分工？	NOT NULL 拒绝缺值；UNIQUE 拒绝重复；CHECK 检查表达式；外键检查对应目标。	逐渐认熟并按坏值定位，不能用其中一个替代其余。单独 UNIQUE 不拒绝多个 NULL。
CHECK 和默认值为什么补不出可靠事实？	CHECK 结果为 NULL 时不算违反，因此必填仍需 NOT NULL；DEFAULT 处理省略列，不会把显式 NULL 自动变成初值。	**未知电话保留 NULL，确定的初始状态才用默认值。** 普通文字主键也按本章显式加 NOT NULL。
一个时段最多一条预约，是外键保证的吗？	`slot_id` 外键保证时段存在，UNIQUE 限制它最多被一条预约引用；取消历史也在唯一范围内。	认清“存在”与“数量”是两条规则；此模型没有自动支持取消后复用时段。
开启了约束，就能保证日期和联系信息可靠吗？	日期 TEXT 非空未验证日期、营业时间或重叠；电话长度不证明能拨通；技能表也未证明本次匹配正确。	看实际表达式承诺的范围；日期解析、邮箱规范化、权限和技能匹配仍需应用明确处理。

SQLite 外键开关属于每条连接。需要在事务外设置 `PRAGMA foreign_keys = ON`，再用 `PRAGMA foreign_keys` 读回 1；只在建库时开过一次，不保证以后新连接也检查。完整连接设置可以查，**必须实际启用**的关系不能漏。

`ON DELETE RESTRICT` 保护被引用的一端：仍有工单引用 C1 时，不允许删掉 C1。C04 会把它与取消和级联删除联系起来。

回看：「数据库怎样发现 C999 根本不存在？」「一列写着 INTEGER，为什么还能收下文字？」「哪些事实必须有值，哪些未知应该留下？」「把这些决定放在一起，表应该怎样建？」。

规则写下了，怎样判断已有数据真的受保护？

当前问题	关系或短用法	记到什么程度
出现约束错误，证明到哪一步？	坏值应在预期规则处被拒绝，还要读回或比较状态，确认没有留下新行。	查用种子和验证程序，理解错误类别与数据结果；极小合成库的 `iterdump()` 比较不能照搬为真实业务日志。
修改 CREATE TABLE 文件，会更新旧库吗？	不会；它只改变以后怎样建表。迁移要让旧结构与旧行一起进入新规则。	**改定义文件与迁移已有数据库分开。** 新增可空列时，旧记录不知道的新事实仍可为 NULL。
旧数据不满足新 CHECK，怎样处理？	先查出违例行，核对含义；复制到新表若失败，明确回滚并读回旧表和新表状态。	不静默丢行、不猜修正值；一条复制语句失败不等于整次迁移已撤销。
两列表复制成功，就能正式替换业务表吗？	真实迁移还涉及引用它的表、view、应用、外键与恢复副本；小实验只演练复制和撤回。	长迁移步骤按目标版本查，在副本核对完整行与依赖；不把演练成功写成正式切换完成。

原章 `c_schema.sql`、`c_seed.sql` 与 `c03_check.py` 使用独立内存库。成功插入 A1 只证明这一行满足结构约束，尚未同步占用时段；C04 才把预约、时段与事件纳入同一个完成条件。

回看：「怎样知道约束真的在工作？」「已经存了旧数据，新规则怎么加进去？」。